Automation

Context-As a Automation Architect generate me Selenium Java framework structure

Instruction

-ASK me 5 questions one by one for framework creation

-Ask me the next question after me providing the answer

-Framework should be Entrperise following the industry standard

Step by step implementation

I'll act as your Automation Architect and guide you through building an enterprise-grade Selenium Java framework following industry standards (e.g., Page Object Model, Singleton WebDriver, Factory pattern, Logging, Reporting, Data-driven, CI/CD readiness).

Let me begin with Question 1 of 5:

Question 1:

What is your project/product name and primary application under test (AUT) type?

Please specify if it's a:

Public web application (e.g., e-commerce, banking)

Internal enterprise portal (e.g., CRM, ERP, HRMS)

Hybrid with iframes, multiple windows, or shadow DOM

Also, mention the base URL (if available) or core domain.

Hybrid with iframes, multiple windows, or shadow DOM

Thank you for clarifying the AUT type — Hybrid with iframes, multiple windows, and shadow DOM.

This means our framework must include dedicated handlers for shadow roots, window switching, and frame management.

Let me capture that and move to Question 2 of 5:

Question 2:

Which test data source will your framework use for different environments (Dev, QA, Staging, Prod)?

Please choose one or specify your own:

A) Properties file (config.properties) + Excel (test data)

B) JSON / YAML files

C) Database (JDBC)

D) Apache POI + TestNG DataProvider with external Excel

E) Environment-specific config + Java Faker for dynamic data

Also, do you need data segregation per environment (e.g., QA URLs, credentials, API keys)?

Properties file (config.properties) + Excel (test data)

Perfect. Using config.properties for configuration and Excel for test data is a robust, industry-standard choice — easy to maintain and environment-agnostic.

Let me proceed to Question 3 of 5:

Question 3:

Which reporting and logging framework would you like to integrate for enterprise-level visibility and debugging?

Please choose one combination:

A) ExtentReports (with Spark/HTML) + Log4j2

B) Allure Report + SLF4J with Logback

C) ReportNG + Java Util Logging

D) Custom HTML report using TestNG listeners + Log4j2

Also, specify if you need:

Screenshot capture on test failure

Logs attached to reports

Dashboard for historical trends (yes/no)

ExtentReports (with Spark/HTML) + Log4j2

creenshot capture on test failure

Logs attached to reports

Dashboard for historical trends (yes)

Excellent choice. ExtentReports (Spark) + Log4j2 with screenshots on failure, logs in reports, and historical dashboard gives you enterprise-grade reporting and debugging capabilities.

Let's move to Question 4 of 5:

Question 4:

How should the framework handle test execution flow and browser management for parallel testing?

Please specify:

A) Execution approach:

Maven surefire + TestNG XML (suite-based)

Jenkins / GitHub Actions parameterized run

Cross-browser via command line (browser=chrome/firefox/edge)

B) Parallel execution scope:

Methods (multiple @Test within a class)

Classes (multiple test classes)

Suites (multiple suites)

C) WebDriver management:

Local WebDriver Manager (e.g., WebDriverManager by Bonigarcia)

Docker Selenium Grid (Zalenium / Selenoid)

Cloud (BrowserStack / SauceLabs / LambdaTest)

D) Singleton WebDriver or ThreadLocal<WebDriver> for thread safety? (Mandatory for parallel execution)

Please provide your choices for A, B, C, and D.

A) Execution approach: -Jenkins / GitHub Actions parameterized run

B) Parallel execution scope: Suites (multiple suites)

C) WebDriver management: Cloud (BrowserStack)

D) ThreadLocal<WebDriver>

Perfect. Jenkins/GitHub Actions + Suite-level parallel + BrowserStack cloud + ThreadLocal<WebDriver> — this is an enterprise-grade, scalable, parallel-ready setup.

Now for the final Question 5 of 5:

Question 5:

What are your team's skill level and CI/CD trigger strategy, and do you have any specific compliance or naming standards?

Please specify:

A) Team expertise:

Junior (needs detailed logging, self-healing locators, retry mechanism)

Mid-level (standard POM + utilities)

Senior (custom annotations, listeners, DSL layer)

B) CI/CD trigger:

On push to specific branch (e.g., develop, main)

Poll SCM (nightly batch)

Manual trigger with parameters (browser, environment, suite name)

Webhook (e.g., on PR merge)

C) Naming conventions & compliance:

Test class naming (e.g., LoginTest, *_TC001)

Method naming (e.g., testValidLogin, verifyDashboard)

Package structure (e.g., com.company.project.pages, tests, utils)

Any compliance needs (GDPR, accessibility testing, data masking in logs)

D) Additional requirements:

Retry failed tests (yes/no, max attempts)

Email report on completion (yes/no)

Environment-specific properties override (yes/no)

Please provide your answers for A, B, C, and D.

A) Team expertise: Junior (needs detailed logging, self-healing locators, retry mechanism,standard POM + utilities)

B) CI/CD trigger:Poll SCM (nightly batch) Manual trigger with parameters (browser, environment, suite name)

C) Naming conventions & compliance: All naming conventions must be followed also having accessibility testing feature

D) Additional requirements:

Retry failed tests (yes)

Email report on completion (yes)

Environment-specific properties override (yes)

Thank you for providing all the inputs. As your Automation Architect, I will now generate a complete enterprise-grade Selenium Java framework structure tailored to your exact needs.

Framework Overview

Framework Name: HybridEnterprise-Automation

Key Features:

Hybrid application support (iframes, windows, shadow DOM)

Page Object Model + Utilities + Self-Healing Locators

ExtentReports (Spark) + Log4j2 with screenshots & historical dashboard

ThreadLocal WebDriver + BrowserStack Cloud + Suite-level parallel execution

Jenkins/GitHub Actions (Poll SCM + Manual parameterized)

Retry mechanism + Email reports + Environment-specific overrides

Accessibility testing (Axe-core integration)

Project Structure

text

```
HybridEnterprise-Automation/
│
├── pom.xml
├── testng.xml
├── README.md
├── .gitignore
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── company/
│   │   │           └── automation/
│   │   │               ├── core/
```

```
│ │ │        │  ├── DriverManager.java       (ThreadLocal WebDriver)
│ │ │        │  ├── BrowserStackConfig.java    (Cloud setup)
│ │ │        │  ├── PropertyManager.java      (config.properties)
│ │ │        │  └── EnvironmentConfig.java     (Env override)
│ │ │        │
│ │ │        ├── pages/
│ │ │        │  ├── BasePage.java           (Common methods + Self-Healing)
│ │ │        │  ├── LoginPage.java
│ │ │        │  ├── DashboardPage.java
│ │ │        │  └── ... (other POM classes)
│ │ │        │
│ │ │        ├── utils/
│ │ │        │  ├── ExcelReader.java         (Apache POI)
│ │ │        │  ├── Log4j2Manager.java
│ │ │        │  ├── ScreenshotUtil.java
│ │ │        │  ├── EmailUtil.java           (Email report)
│ │ │        │  ├── RetryAnalyzer.java        (Test retry)
│ │ │        │  ├── AccessibilityUtil.java     (Axe-core)
│ │ │        │  ├── ShadowDOMHandler.java
│ │ │        │  ├── WindowHandler.java
│ │ │        │  ├── FrameHandler.java
│ │ │        │  └── SelfHealingLocator.java    (Fallback strategies)
│ │ │        │
│ │ │        ├── listeners/
│ │ │        │  ├── ExtentReportListener.java   (Spark + dashboard)
│ │ │        │  ├── TestExecutionListener.java
│ │ │        │  └── AnnotationTransformer.java  (Retry via annotation)
```

```
│ │ │          │
│ │ │          ├── exceptions/
│ │ │          │   ├── ElementNotFoundException.java
│ │ │          │   └── FrameworkException.java
│ │ │          │
│ │ │          └── constants/
│ │ │              ├── FrameworkConstants.java
│ │ │              └── Enums.java            (Browser, Environment)
│ │ │
│ │ └── resources/
│ │     ├── config/
│ │     │   ├── config.properties              (Main config)
│ │     │   ├── dev-config.properties
│ │     │   ├── qa-config.properties
│ │     │   └── staging-config.properties
│ │     ├── log4j2.xml
│ │     ├── extent-config.xml
│ │     └── accessibility-config.json
│ │
│ └── test/
│     ├── java/
│     │   └── com/
│     │       └── company/
│     │           └── automation/
│     │               ├── tests/
│     │               │   ├── base/
│     │               │   │   └── BaseTest.java        (Setup/Teardown)
```

```
│   │       │   ├── login/
│   │       │   │   └── LoginTest.java
│   │       │   ├── dashboard/
│   │       │   │   └── DashboardTest.java
│   │       │   └── accessibility/
│   │       │       └── AccessibilityTest.java
│   │       │
│   │       └── dataproviders/
│   │           └── ExcelDataProvider.java
│   │
│   └── resources/
│       ├── testdata/
│       │   ├── login_data.xlsx
│       │   └── user_data.xlsx
│       └── suites/
│           ├── smoke_suite.xml
│           ├── regression_suite.xml
│           └── accessibility_suite.xml
│
├── test-output/
│   ├── extent-reports/
│   │   ├── spark-report.html
│   │   └── historical-dashboard/
│   └── screenshots/
│
├── logs/
│   └── application.log
```

```
│
├── Jenkinsfile                    (Pipeline for Poll SCM + Manual)
└── .github/workflows/maven.yml    (GitHub Actions fallback)
```

Step-by-Step Implementation

Step 1: Create Maven Project & Update pom.xml

xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.company</groupId>
  <artifactId>HybridEnterprise-Automation</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suiteFile>testng.xml</suiteFile>
    <browser>chrome</browser>
    <environment>qa</environment>
  </properties>

  <dependencies>
    <!-- Selenium -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
```

```
    <artifactId>selenium-java</artifactId>
    <version>4.15.0</version>
</dependency>

<!-- WebDriverManager (Local fallback) -->
<dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.6.2</version>
</dependency>

<!-- BrowserStack SDK -->
<dependency>
    <groupId>com.browserstack</groupId>
    <artifactId>browserstack-java-sdk</artifactId>
    <version>1.15.0</version>
</dependency>

<!-- TestNG -->
<dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.8.0</version>
    <scope>test</scope>
</dependency>

<!-- ExtentReports -->
<dependency>
```

```
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.1</version>
</dependency>

<!-- Log4j2 -->
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.20.0</version>
</dependency>
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.20.0</version>
</dependency>

<!-- Apache POI (Excel) -->
<dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.5</version>
</dependency>

<!-- Java Faker (Dynamic test data) -->
<dependency>
    <groupId>com.github.javafaker</groupId>
    <artifactId>javafaker</artifactId>
```

```
        <version>1.0.2</version>
    </dependency>

    <!-- Accessibility Testing (Axe-core) -->
    <dependency>
        <groupId>com.deque</groupId>
        <artifactId>axe-selenium</artifactId>
        <version>3.0</version>
    </dependency>

    <!-- Email (Jakarta Mail) -->
    <dependency>
        <groupId>com.sun.mail</groupId>
        <artifactId>jakarta.mail</artifactId>
        <version>2.0.1</version>
    </dependency>

    <!-- JSON Simple -->
    <dependency>
        <groupId>com.googlecode.json-simple</groupId>
        <artifactId>json-simple</artifactId>
        <version>1.1.1</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
```

```
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.0.0-M9</version>
            <configuration>
              <suiteXmlFiles>
                <suiteXmlFile>${suiteFile}</suiteXmlFile>
              </suiteXmlFiles>
              <systemPropertyVariables>
                <browser>${browser}</browser>
                <environment>${environment}</environment>
              </systemPropertyVariables>
            </configuration>
          </plugin>
        </plugins>
    </build>
</project>
```

Step 2: Core Components

DriverManager.java (ThreadLocal + BrowserStack)

java

```
package com.company.automation.core;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import com.browserstack.local.Local;
import io.github.bonigarcia.wdm.WebDriverManager;
import java.util.HashMap;
```

```
public class DriverManager {
   private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();
   private static Local bsLocal;

   public static void initDriver(String browser, String environment) {
      WebDriver webDriver = null;
      String executionEnv = PropertyManager.getProperty("execution.env"); // cloud or
local

      if (executionEnv.equalsIgnoreCase("cloud")) {
         // BrowserStack configuration
         HashMap<String, String> caps = new HashMap<>();
         caps.put("browserName", browser);
         caps.put("browserstack.user", PropertyManager.getProperty("bs.username"));
         caps.put("browserstack.key", PropertyManager.getProperty("bs.accesskey"));
         caps.put("browserstack.local", "true");
         caps.put("build", "HybridEnterprise-Build");
         caps.put("name", "Test Suite Execution");

         // Start BrowserStack Local for internal apps
         bsLocal = new Local();
         HashMap<String, String> bsLocalArgs = new HashMap<>();
         bsLocalArgs.put("key", PropertyManager.getProperty("bs.accesskey"));
         try {
            bsLocal.start(bsLocalArgs);
         } catch (Exception e) {
            e.printStackTrace();
```

```
        }

        webDriver = new RemoteWebDriver(new URL("https://hub-
cloud.browserstack.com/wd/hub"), caps);
      } else {
        // Local execution with WebDriverManager
        switch (browser.toLowerCase()) {
          case "chrome":
            webDriver = WebDriverManager.chromedriver().create();
            break;
          case "firefox":
            webDriver = WebDriverManager.firefoxdriver().create();
            break;
          case "edge":
            webDriver = WebDriverManager.edgedriver().create();
            break;
          default:
            throw new FrameworkException("Unsupported browser: " + browser);
        }
      }

      driver.set(webDriver);
      driver.get().manage().window().maximize();
      driver.get().manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
    }

    public static WebDriver getDriver() {
      return driver.get();
```

```
    }

    public static void quitDriver() {
        if (driver.get() != null) {
            driver.get().quit();
            driver.remove();
        }
        if (bsLocal != null && bsLocal.isRunning()) {
            bsLocal.stop();
        }
    }
}
```

SelfHealingLocator.java (For Junior Team)

java

```
package com.company.automation.utils;

import org.openqa.selenium.*;
import java.util.*;

public class SelfHealingLocator {

    public static WebElement findElement(WebDriver driver, By originalBy, String
elementName) {
        try {
            return driver.findElement(originalBy);
        } catch (NoSuchElementException e) {
            Log4j2Manager.warn("Original locator failed for: " + elementName + ". Attempting
self-healing...");
```

```
        // Strategy 1: Try partial attributes
        List<WebElement> candidates = driver.findElements(By.xpath("//*[contains(@id, '" + getLastPart(originalBy) + "') or contains(@class, '" + getLastPart(originalBy) + "')]"));

        // Strategy 2: Try text-based
        if (candidates.isEmpty()) {
            candidates = driver.findElements(By.xpath("//*[text()='" + elementName + "' or contains(text(), '" + elementName + "')]"));
        }

        // Strategy 3: Try aria-label
        if (candidates.isEmpty()) {
            candidates = driver.findElements(By.xpath("//*[@aria-label='" + elementName + "']"));
        }

        if (!candidates.isEmpty()) {
            Log4j2Manager.info("Self-healing succeeded for: " + elementName);
            return candidates.get(0);
        }
        throw new FrameworkException("Self-healing failed for: " + elementName);
    }
  }

  private static String getLastPart(By by) {
    String byString = by.toString();
    return byString.substring(byString.lastIndexOf(":") + 1).replace("']", "").replace("'", "");
  }
}
```

Step 3: BasePage with Handlers

java

```java
package com.company.automation.pages;

import com.company.automation.core.DriverManager;
import com.company.automation.utils.*;
import org.openqa.selenium.*;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.*;
import java.time.Duration;

public abstract class BasePage {
    protected WebDriver driver;
    protected WebDriverWait wait;
    protected Actions actions;
    protected JavascriptExecutor js;
    protected ShadowDOMHandler shadowHandler;
    protected WindowHandler windowHandler;
    protected FrameHandler frameHandler;

    public BasePage() {
        this.driver = DriverManager.getDriver();
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        this.actions = new Actions(driver);
        this.js = (JavascriptExecutor) driver;
        this.shadowHandler = new ShadowDOMHandler(driver);
        this.windowHandler = new WindowHandler(driver);
```

```
        this.frameHandler = new FrameHandler(driver);
        PageFactory.initElements(driver, this);
    }

    // Self-healing click with retry
    protected void safeClick(By locator, String elementName) {
        try {
            wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
            Log4j2Manager.info("Clicked on: " + elementName);
        } catch (Exception e) {
            Log4j2Manager.warn("Standard click failed. Attempting self-healing click...");
            WebElement element = SelfHealingLocator.findElement(driver, locator,
elementName);
            element.click();
        }
    }

    protected void safeSendKeys(By locator, String text, String elementName) {
        WebElement element = SelfHealingLocator.findElement(driver, locator,
elementName);
        element.clear();
        element.sendKeys(text);
        Log4j2Manager.info("Entered text in: " + elementName);
    }

    // Shadow DOM support
    protected WebElement findInShadowRoot(By shadowHostLocator, By targetLocator) {
        return shadowHandler.findElement(shadowHostLocator, targetLocator);
    }
```

```
    // Switch to frame
    protected void switchToFrame(By frameLocator) {
        frameHandler.switchToFrame(frameLocator);
    }

    // Switch to new window
    protected void switchToNewWindow() {
        windowHandler.switchToNewWindow();
    }
}
```

Step 4: Test with Retry & DataProvider

java

```
package com.company.automation.tests.login;

import com.company.automation.tests.base.BaseTest;
import com.company.automation.pages.LoginPage;
import com.company.automation.dataproviders.ExcelDataProvider;
import com.company.automation.utils.RetryAnalyzer;
import org.testng.annotations.Test;
import org.testng.Assert;

public class LoginTest extends BaseTest {

    @Test(dataProvider = "getLoginData", dataProviderClass = ExcelDataProvider.class,
        retryAnalyzer = RetryAnalyzer.class, groups = {"smoke", "regression"})
    public void testValidLogin(String username, String password, String expectedTitle) {
        LoginPage loginPage = new LoginPage();
```

```
        loginPage.enterUsername(username);
        loginPage.enterPassword(password);
        loginPage.clickLogin();

        String actualTitle = driver.getTitle();
        Assert.assertEquals(actualTitle, expectedTitle, "Login failed or title mismatch");
        Log4j2Manager.info("Login test passed for user: " + username);
    }
}
```

Step 5: Jenkins Pipeline (Jenkinsfile)

groovy

```
pipeline {
    agent any

    parameters {
        choice(name: 'BROWSER', choices: ['chrome', 'firefox', 'edge'], description: 'Select browser')
        choice(name: 'ENVIRONMENT', choices: ['dev', 'qa', 'staging', 'prod'], description: 'Select environment')
        choice(name: 'SUITE', choices: ['smoke_suite.xml', 'regression_suite.xml', 'accessibility_suite.xml'], description: 'Test suite')
    }

    triggers {
        pollSCM('H 2 * * *')  // Nightly batch at 2 AM
    }

    environment {
        BROWSER_STACK_USER = credentials('browserstack-user')
```

```
    BROWSER_STACK_ACCESS_KEY = credentials('browserstack-key')
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Run Tests') {
      steps {
        script {
          sh "mvn clean test -Dbrowser=${params.BROWSER} -
Denvironment=${params.ENVIRONMENT} -DsuiteFile=${params.SUITE}"
        }
      }
    }

    stage('Generate Report & Email') {
      steps {
        script {
          sh "python3 send_email_report.py"  // EmailUtil can be invoked here
        }
      }
    }
  }
```

```
  post {
    always {
      publishHTML([
        reportDir: 'test-output/extent-reports/',
        reportFiles: 'spark-report.html',
        reportName: 'Extent Report'
      ])
    }
  }
}
```

Step 6: Accessibility Testing Integration

java

```
package com.company.automation.utils;

import com.deque.html.axecore.results.Results;
import com.deque.html.axecore.selenium.AxeBuilder;
import org.json.JSONArray;
import org.json.JSONObject;

public class AccessibilityUtil {

  public static void runAccessibilityTest(WebDriver driver, String pageName) {
    AxeBuilder axeBuilder = new AxeBuilder();
    Results results = axeBuilder.analyze(driver);

    JSONArray violations = new JSONArray(results.getViolations());
    if (violations.length() > 0) {
```

```
        Log4j2Manager.error("Accessibility violations found on " + pageName + ": " +
violations.toString());

        ScreenshotUtil.captureScreenshot(driver, pageName + "_a11y_failure");

        throw new FrameworkException("Accessibility test failed with " +
violations.length() + " violations");

      } else {

        Log4j2Manager.info("Accessibility test passed for: " + pageName);

      }

   }

}
```

Step 7: Configuration Files

config.properties (Main):

```
properties

execution.env=cloud

bs.username=${BROWSER_STACK_USER}

bs.accesskey=${BROWSER_STACK_ACCESS_KEY}

implicit.wait=10

explicit.wait=15

screenshot.on.failure=true

retry.count=2

email.report.to=qa-team@company.com
```

qa-config.properties (Environment override):

```
properties

base.url=https://qa-application.company.com

default.username=qa_user

default.password=Qa@2024
```

Framework Highlights for Your Junior Team

Feature Implementation

Self-healing locators SelfHealingLocator.java with fallback strategies

Detailed logging Log4j2 with INFO, WARN, ERROR levels

Retry mechanism RetryAnalyzer.java with 2 attempts

Standard POM BasePage.java + individual page classes

Accessibility Axe-core integration in AccessibilityUtil

Email reports EmailUtil.java with HTML attachment

Environment override PropertyManager loads env-specific config

Parallel suites TestNG XML + ThreadLocal

Next Steps for You

Clone this structure into your IDE (IntelliJ/Eclipse)

Update BrowserStack credentials in Jenkins/GitHub Secrets

Create Excel files under src/test/resources/testdata/ with your test data

Run locally first: mvn clean test -Dbrowser=chrome -Denvironment=qa -DsuiteFile=smoke_suite.xml

Set up Jenkins with Poll SCM (nightly) + parameterized builds

Would you like me to provide the complete code for any specific utility class (e.g., ExcelReader, EmailUtil, ShadowDOMHandler) or help you with the TestNG XML configuration for suite-level parallel execution?

yes

Excellent. I'll provide you with the complete code for the most critical utility classes and the TestNG XML configuration for suite-level parallel execution.

1. Complete ExcelReader Utility Class

java

```java
package com.company.automation.utils;

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class ExcelReader {
    private static Workbook workbook;
    private static Sheet sheet;
    private static Cell cell;
    private static Row row;

    /**
     * Reads test data from Excel file and returns as 2D Object array for TestNG DataProvider
     * @param filePath Excel file path
     * @param sheetName Sheet name
     * @return Object[][] data
     */
    public static Object[][] getTestData(String filePath, String sheetName) {
        Object[][] data = null;

        try (FileInputStream fis = new FileInputStream(filePath)) {
```

```
        workbook = new XSSFWorkbook(fis);
        sheet = workbook.getSheet(sheetName);

        int rowCount = sheet.getPhysicalNumberOfRows();
        int colCount = sheet.getRow(0).getPhysicalNumberOfCells();

        data = new Object[rowCount - 1][colCount];

        for (int i = 1; i < rowCount; i++) {
            row = sheet.getRow(i);
            for (int j = 0; j < colCount; j++) {
                cell = row.getCell(j);
                data[i - 1][j] = getCellValue(cell);
            }
        }
    } catch (IOException e) {
        Log4j2Manager.error("Failed to read Excel file: " + filePath, e);
        throw new FrameworkException("Excel file not found or corrupted: " + filePath);
    } finally {
        try {
            if (workbook != null) workbook.close();
        } catch (IOException e) {
            Log4j2Manager.warn("Failed to close workbook", e);
        }
    }
    return data;
}
```

```
/**
 * Reads data as Map for row-wise operations
 */
public static Map<String, String> getTestDataAsMap(String filePath, String
sheetName, int rowNumber) {
    Map<String, String> dataMap = new HashMap<>();

    try (FileInputStream fis = new FileInputStream(filePath)) {
        workbook = new XSSFWorkbook(fis);
        sheet = workbook.getSheet(sheetName);

        Row headerRow = sheet.getRow(0);
        Row dataRow = sheet.getRow(rowNumber);

        for (int j = 0; j < headerRow.getPhysicalNumberOfCells(); j++) {
            String key = getCellValue(headerRow.getCell(j));
            String value = getCellValue(dataRow.getCell(j));
            dataMap.put(key, value);
        }
    } catch (IOException e) {
        Log4j2Manager.error("Failed to read Excel file as map", e);
    }
    return dataMap;
}

/**
 * Writes data back to Excel (for reporting or updating test data)
 */
```

```
    public static void writeToExcel(String filePath, String sheetName, int rowNum, int
colNum, String value) {
        try (FileInputStream fis = new FileInputStream(filePath)) {
            workbook = new XSSFWorkbook(fis);
            sheet = workbook.getSheet(sheetName);

            row = sheet.getRow(rowNum);
            if (row == null) row = sheet.createRow(rowNum);

            cell = row.getCell(colNum);
            if (cell == null) cell = row.createCell(colNum);

            cell.setCellValue(value);

            try (FileOutputStream fos = new FileOutputStream(filePath)) {
                workbook.write(fos);
            }
        } catch (IOException e) {
            Log4j2Manager.error("Failed to write to Excel", e);
        }
    }

    private static String getCellValue(Cell cell) {
        if (cell == null) return "";

        switch (cell.getCellType()) {
            case STRING:
                return cell.getStringCellValue();
```

```
        case NUMERIC:
          if (DateUtil.isCellDateFormatted(cell)) {
            return cell.getDateCellValue().toString();
          } else {
            return String.valueOf((long) cell.getNumericCellValue());
          }
        case BOOLEAN:
          return String.valueOf(cell.getBooleanCellValue());
        case FORMULA:
          return cell.getCellFormula();
        default:
          return "";
      }
    }
}
```

2. Complete EmailUtil Class (with HTML Report Attachment)

java

```
package com.company.automation.utils;

import javax.activation.DataHandler;
import javax.activation.DataSource;
import javax.activation.FileDataSource;
import javax.mail.*;
import javax.mail.internet.*;
import java.io.File;
import java.util.Properties;

public class EmailUtil {
```

```
    private static final String SMTP_HOST = PropertyManager.getProperty("smtp.host");
    private static final String SMTP_PORT = PropertyManager.getProperty("smtp.port");
    private static final String FROM_EMAIL = PropertyManager.getProperty("from.email");
    private static final String FROM_PASSWORD =
PropertyManager.getProperty("from.password");
    private static final String TO_EMAIL = PropertyManager.getProperty("email.report.to");

    /**
     * Sends email with ExtentReport attachment
     * @param subject Email subject
     * @param body Email body content
     * @param reportPath Path to HTML report
     */
    public static void sendEmailWithReport(String subject, String body, String reportPath)
{
        Properties props = new Properties();
        props.put("mail.smtp.host", SMTP_HOST);
        props.put("mail.smtp.port", SMTP_PORT);
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");

        Session session = Session.getInstance(props, new Authenticator() {
            @Override
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(FROM_EMAIL, FROM_PASSWORD);
            }
        });
```

```
    try {
        Message message = new MimeMessage(session);
        message.setFrom(new InternetAddress(FROM_EMAIL));
        message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(TO_EMAIL));
        message.setSubject(subject);

        // Create HTML content
        MimeBodyPart textPart = new MimeBodyPart();
        textPart.setContent(body, "text/html");

        // Attach report
        MimeBodyPart attachmentPart = new MimeBodyPart();
        DataSource source = new FileDataSource(reportPath);
        attachmentPart.setDataHandler(new DataHandler(source));
        attachmentPart.setFileName(new File(reportPath).getName());

        Multipart multipart = new MimeMultipart();
        multipart.addBodyPart(textPart);
        multipart.addBodyPart(attachmentPart);

        message.setContent(multipart);
        Transport.send(message);

        Log4j2Manager.info("Email sent successfully to: " + TO_EMAIL);
    } catch (MessagingException e) {
        Log4j2Manager.error("Failed to send email", e);
    }
```

```
    }

    /**
     * Generates HTML summary for email body
     */
    public static String generateEmailBody(int totalTests, int passed, int failed, int skipped, double passPercentage) {
        return "<html>" +
                "<body style='font-family: Arial, sans-serif;'>" +
                "<h2 style='color: #2c3e50;'>Automation Test Execution Report</h2>" +
                "<table border='1' cellpadding='10' style='border-collapse: collapse;'>" +
                "<tr bgcolor='#3498db'><th style='color: white;'>Metric</th><th style='color: white;'>Value</th></tr>" +
                "<tr><td>Total Tests</td><td><b>" + totalTests + "</b></td></tr>" +
                "<tr bgcolor='#d5f5e3'><td>Passed</td><td style='color: green;'><b>" + passed + "</b></td></tr>" +
                "<tr bgcolor='#fadbd8'><td>Failed</td><td style='color: red;'><b>" + failed + "</b></td></tr>" +
                "<tr><td>Skipped</td><td><b>" + skipped + "</b></td></tr>" +
                "<tr bgcolor='#d4e6f1'><td>Pass Percentage</td><td><b>" + passPercentage + "%</b></td></tr>" +
                "</table>" +
                "<p>Please find the detailed Extent Report attached.</p>" +
                "<p>Regards,<br/>Automation Team</p>" +
                "</body></html>";
    }
}
```

3. Complete ShadowDOMHandler Class

java

```
package com.company.automation.utils;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;
import java.util.List;
import java.util.stream.Collectors;

public class ShadowDOMHandler {
  private WebDriver driver;
  private WebDriverWait wait;

  public ShadowDOMHandler(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
  }

  /**
   * Finds element inside shadow DOM (nested support)
   * @param shadowHostLocator Locator for shadow host element
   * @param targetLocator Locator for target element inside shadow root
   */
  public WebElement findElement(By shadowHostLocator, By targetLocator) {
    WebElement shadowHost = driver.findElement(shadowHostLocator);
    SearchContext shadowRoot = getShadowRoot(shadowHost);
    return shadowRoot.findElement(targetLocator);
  }
```

```
/**
 * Handles nested shadow DOM (multiple levels)
 * @param locators List of locators for each shadow level
 */
public WebElement findElementInNestedShadow(List<By> locators) {
   SearchContext currentContext = driver;

   for (int i = 0; i < locators.size() - 1; i++) {
      WebElement shadowHost = currentContext.findElement(locators.get(i));
      currentContext = getShadowRoot(shadowHost);
   }

   return currentContext.findElement(locators.get(locators.size() - 1));
}

/**
 * Finds multiple elements inside shadow DOM
 */
public List<WebElement> findElements(By shadowHostLocator, By targetLocator) {
   WebElement shadowHost = driver.findElement(shadowHostLocator);
   SearchContext shadowRoot = getShadowRoot(shadowHost);
   return shadowRoot.findElements(targetLocator);
}

/**
 * Clicks element inside shadow DOM with retry
 */
```

```
    public void clickShadowElement(By shadowHostLocator, By targetLocator, String
elementName) {
        try {
            WebElement element = findElement(shadowHostLocator, targetLocator);
            wait.until(ExpectedConditions.elementToBeClickable(element)).click();
            Log4j2Manager.info("Clicked on shadow DOM element: " + elementName);
        } catch (Exception e) {
            Log4j2Manager.error("Failed to click shadow DOM element: " + elementName, e);
            throw new FrameworkException("Shadow DOM interaction failed: " +
elementName);
        }
    }

    /**
     * Types text into shadow DOM input field
     */
    public void typeInShadowElement(By shadowHostLocator, By targetLocator, String
text, String elementName) {
        WebElement element = findElement(shadowHostLocator, targetLocator);
        element.clear();
        element.sendKeys(text);
        Log4j2Manager.info("Entered text in shadow DOM field: " + elementName);
    }

    /**
     * Gets shadow root using JavaScript (cross-browser compatible)
     */
    private SearchContext getShadowRoot(WebElement shadowHost) {
        try {
```

```
        return (SearchContext) ((JavascriptExecutor) driver).executeScript(
            "return arguments[0].shadowRoot", shadowHost);
    } catch (Exception e) {
        Log4j2Manager.error("Failed to access shadow root", e);
        throw new FrameworkException("Cannot access shadow DOM. Might be closed
or not available.");
    }
  }

  /**
   * Checks if shadow root is open (accessible)
   */
  public boolean isShadowRootOpen(WebElement shadowHost) {
    try {
        Object shadowRoot = ((JavascriptExecutor) driver).executeScript(
            "return arguments[0].shadowRoot", shadowHost);
        return shadowRoot != null;
    } catch (Exception e) {
        return false;
    }
  }
}
```

4. Complete WindowHandler Class

java

```
package com.company.automation.utils;

import org.openqa.selenium.WebDriver;
import java.util.ArrayList;
```

```java
import java.util.Set;

public class WindowHandler {
    private WebDriver driver;
    private String mainWindowHandle;

    public WindowHandler(WebDriver driver) {
        this.driver = driver;
        this.mainWindowHandle = driver.getWindowHandle();
    }

    /**
     * Switches to newly opened window (useful for pop-ups)
     */
    public void switchToNewWindow() {
        Set<String> windowHandles = driver.getWindowHandles();
        for (String handle : windowHandles) {
            if (!handle.equals(mainWindowHandle)) {
                driver.switchTo().window(handle);
                Log4j2Manager.info("Switched to new window with title: " + driver.getTitle());
                break;
            }
        }
    }

    /**
     * Switches to window by title (partial match)
     */
```

```
    public void switchToWindowByTitle(String partialTitle) {
        Set<String> windowHandles = driver.getWindowHandles();
        for (String handle : windowHandles) {
            driver.switchTo().window(handle);
            if (driver.getTitle().contains(partialTitle)) {
                Log4j2Manager.info("Switched to window with title containing: " + partialTitle);
                return;
            }
        }
        Log4j2Manager.warn("No window found with title containing: " + partialTitle);
    }

    /**
     * Switches to window by index (0 = main, 1 = first popup, etc.)
     */
    public void switchToWindowByIndex(int index) {
        Set<String> windowHandles = driver.getWindowHandles();
        ArrayList<String> handlesList = new ArrayList<>(windowHandles);

        if (index < handlesList.size()) {
            driver.switchTo().window(handlesList.get(index));
            Log4j2Manager.info("Switched to window at index: " + index);
        } else {
            throw new FrameworkException("Window index " + index + " not available. Total
windows: " + handlesList.size());
        }
    }
```

```
    /**
     * Closes current window and switches back to main window
     */
    public void closeCurrentWindowAndSwitchToMain() {
        driver.close();
        driver.switchTo().window(mainWindowHandle);
        Log4j2Manager.info("Closed current window, switched back to main window");
    }

    /**
     * Returns all window handles
     */
    public ArrayList<String> getAllWindowHandles() {
        return new ArrayList<>(driver.getWindowHandles());
    }

    /**
     * Refreshes main window handle (call after opening new windows)
     */
    public void refreshMainWindowHandle() {
        this.mainWindowHandle = driver.getWindowHandle();
    }
}
```

5. Complete FrameHandler Class

java

```
package com.company.automation.utils;

import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

public class FrameHandler {
    private WebDriver driver;

    public FrameHandler(WebDriver driver) {
        this.driver = driver;
    }

    /**
     * Switches to frame using WebElement
     */
    public void switchToFrame(WebElement frameElement) {
        driver.switchTo().frame(frameElement);
        Log4j2Manager.info("Switched to frame: " + frameElement.toString());
    }

    /**
     * Switches to frame using index
     */
    public void switchToFrame(int index) {
        driver.switchTo().frame(index);
        Log4j2Manager.info("Switched to frame at index: " + index);
    }

    /**
     * Switches to frame using name or ID
```

```
 */
public void switchToFrame(String nameOrId) {
    driver.switchTo().frame(nameOrId);
    Log4j2Manager.info("Switched to frame: " + nameOrId);
}

/**
 * Switches to frame using locator
 */
public void switchToFrame(By frameLocator) {
    WebElement frameElement = driver.findElement(frameLocator);
    driver.switchTo().frame(frameElement);
    Log4j2Manager.info("Switched to frame located by: " + frameLocator.toString());
}

/**
 * Switches back to default content (main page)
 */
public void switchToDefaultContent() {
    driver.switchTo().defaultContent();
    Log4j2Manager.info("Switched back to default content");
}

/**
 * Switches to parent frame (one level up)
 */
public void switchToParentFrame() {
    driver.switchTo().parentFrame();
```

```
        Log4j2Manager.info("Switched to parent frame");
    }

    /**
     * Handles nested frames by switching through multiple levels
     */
    public void switchToNestedFrame(By... frameLocators) {
        switchToDefaultContent();
        for (By locator : frameLocators) {
            WebElement frameElement = driver.findElement(locator);
            driver.switchTo().frame(frameElement);
            Log4j2Manager.info("Switched to nested frame: " + locator.toString());
        }
    }

    /**
     * Returns the total number of frames on the page
     */
    public int getTotalFrameCount() {
        return driver.findElements(By.tagName("iframe")).size() +
               driver.findElements(By.tagName("frame")).size();
    }
}
```

6. Complete TestNG XML for Suite-Level Parallel Execution

testng.xml (Master Suite)

xml

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Enterprise Automation Suite" parallel="suites" thread-count="3">
```

```
  <!-- Suite-level parallel execution: 3 suites will run in parallel -->

  <suite-files>
    <suite-file path="src/test/resources/suites/smoke_suite.xml" />
    <suite-file path="src/test/resources/suites/regression_suite.xml" />
    <suite-file path="src/test/resources/suites/accessibility_suite.xml" />
  </suite-files>

  <listeners>
    <listener class-name="com.company.automation.listeners.ExtentReportListener"
/>
    <listener class-name="com.company.automation.listeners.TestExecutionListener"
/>
  </listeners>

</suite>
```

smoke_suite.xml (Smoke Tests)

xml

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Smoke Test Suite" parallel="classes" thread-count="2">

  <parameter name="suite-type" value="smoke" />

  <listeners>
    <listener class-name="com.company.automation.listeners.ExtentReportListener"
/>
  </listeners>
```

```
  <test name="Login Module Smoke Tests">
    <classes>
      <class name="com.company.automation.tests.login.LoginTest">
        <methods>
          <include name="testValidLogin" />
          <include name="testLoginWithInvalidPassword" />
        </methods>
      </class>
    </classes>
  </test>

  <test name="Dashboard Smoke Tests">
    <classes>
      <class name="com.company.automation.tests.dashboard.DashboardTest">
        <methods>
          <include name="testDashboardWidgets" />
          <include name="testUserProfileDropdown" />
        </methods>
      </class>
    </classes>
  </test>

</suite>
```

regression_suite.xml (Regression with Retry)

xml

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Regression Suite" parallel="classes" thread-count="3">
```

```
  <parameter name="suite-type" value="regression" />

  <test name="Full Regression - All Modules">
    <packages>
      <package name="com.company.automation.tests.*" />
    </packages>
  </test>

</suite>
```

accessibility_suite.xml (A11y Tests)

xml

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Accessibility Test Suite" parallel="none">

  <test name="WCAG 2.1 Compliance Tests">
    <classes>
      <class name="com.company.automation.tests.accessibility.AccessibilityTest">
        <methods>
          <include name="testHomePageAccessibility" />
          <include name="testLoginPageAccessibility" />
          <include name="testDashboardAccessibility" />
        </methods>
      </class>
    </classes>
  </test>

</suite>
```

7. Complete RetryAnalyzer Class

java

```java
package com.company.automation.utils;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class RetryAnalyzer implements IRetryAnalyzer {
    private int retryCount = 0;
    private static final int MAX_RETRY_COUNT = Integer.parseInt(
        PropertyManager.getProperty("retry.count", "2")
    );

    @Override
    public boolean retry(ITestResult result) {
        if (retryCount < MAX_RETRY_COUNT) {
            retryCount++;
            Log4j2Manager.warn("Retrying test '" + result.getName() +
                        "' - Attempt " + retryCount + " of " + MAX_RETRY_COUNT);

            // Capture screenshot on retry
            ScreenshotUtil.captureScreenshot(DriverManager.getDriver(),
                result.getName() + "_retry_" + retryCount);

            return true;
        }
        return false;
    }
}
```

Quick Setup Checklist

Task File/Class

- ✅ Copy ExcelReader utils/ExcelReader.java
- ✅ Copy EmailUtil utils/EmailUtil.java
- ✅ Copy ShadowDOMHandler utils/ShadowDOMHandler.java
- ✅ Copy WindowHandler utils/WindowHandler.java
- ✅ Copy FrameHandler utils/FrameHandler.java
- ✅ Copy RetryAnalyzer utils/RetryAnalyzer.java
- ✅ Configure TestNG XMLs testng.xml + suite files
- ✅ Add SMTP settings config.properties
- ✅ Create Excel test data src/test/resources/testdata/*.xlsx

Do you need me to also provide:

ExcelDataProvider.java (TestNG DataProvider integration with Excel)

ExtentReportListener.java (Complete listener with dashboard history)

ScreenshotUtil.java (With failure capture and automatic naming)

Log4j2Manager.java (Wrapper with log levels and file rotation)

Please let me know which one you'd like next.